ALTERYX DESIGNER

Predicting Shipment Transit Days

A complete step-by-step walkthrough of the predictive analytics workflow built in Alteryx Designer — from raw CSV ingestion to a deployable Random Forest model.

2,000	11	8	2	70 / 30
Shipment Records	Raw Columns	Workflow Steps	Predictive Models	Train / Test Split

WORKFLOW OVERVIEW

Step	Phase	Key Tools	Purpose
1	Data Ingestion	Input Data, Browse	Load CSV, initial profiling
2	Data Cleansing — Formula	Formula	Fix Cost column (? prefix → numeric)
3	Filter Null Delivery Date	Filter	Remove rows where Delivery_Date is null
4	Select — Type Conversion	Select	Change Transit_Days & Cost_Numeric to Double
5	Filter — Remove Outliers	Filter	Keep Transit_Days between 1 and 10
6	Feature Engineering	Formula, Create Samples	Derive 4 new features, 70/30 split
7	Model Training & Scoring	Linear Reg, Random Forest, Score	Train both models on 70% sample
8	Model Comparison & Output	Model Comparison, Output Data	Compare metrics, export results

1 DATA INGESTION

Input Data Tool

The workflow begins by loading the raw logistics CSV file into Alteryx using the Input Data tool. This is the single entry point for all 2,000 shipment records across 11 columns.

logistics_shipments_dataset.csv → Input Data → Browse (profile)

Input Data

IN/OUT PALETTE

Drag from the In/Out palette. Browse to logistics_shipments_dataset.csv. Ensure 'First Row Contains Field Names' is checked. Set Record Limit to 100 for an initial test run, then remove the limit for the full dataset run.

Browse

IN/OUT PALETTE

Connected directly after Input Data. Run the workflow and inspect the Profile tab — this reveals null counts, data types, min/max values, and distinct values for all 11 columns before any transformation is applied.

WHAT THE INITIAL PROFILE REVEALED

The Browse tool showed: (1) The Cost column is a String containing '? ' prefix — it cannot be used as a number until cleaned. (2) Delivery_Date has null values that need filtering. (3) Transit_Days and Cost_Numeric were read as String type due to formatting — they must be explicitly converted to Double. (4) Transit_Days has a maximum of 12, indicating outliers above the normal range.

---

2 DATA CLEANSING — FIX COST COLUMN

Formula Tool — Convert Cost to Numeric

The raw Cost column contains values like ' ? 67.46 ' — a string with a question-mark prefix. We use a Formula tool to strip the '?' and whitespace and create a clean numeric field called Cost_Numeric.

FORMULA EXPRESSION

```
Cost_Numeric (Double) = ToNumber(Trim(Replace([Cost], "?", "")))
```

WHY THIS EXPRESSION WORKS

Replace() removes the '?' character. Trim() strips any remaining leading or trailing whitespace. ToNumber() converts the clean string '67.46' into a usable Double numeric value. Without this step, the Cost field cannot be used as a predictor in any predictive model.

3 FILTER — REMOVE NULL DELIVERY DATES

Filter Tool — IsNull Check

The Delivery_Date column contains null values. Since Transit_Days (our target) is already calculated, rows with a null Delivery_Date are incomplete records that should be excluded from analysis.

After Formula → Filter (IsNull) → True → Exclude → False → Continue

FILTER CONDITION

```
!IsNull([Delivery_Date])
```

How to configure the Filter tool:

- Drag the Filter tool (Preparation palette) after the Formula tool
- In the configuration pane, select 'Custom Filter'
- Enter the expression: !IsNull([Delivery_Date])
- The True (T) output anchor passes valid rows downstream
- Connect a Browse tool to the False (F) anchor to review excluded rows

> **WHY FILTER ON DELIVERY_DATE?**
>
> Rows where Delivery_Date is null are incomplete shipment records — the delivery was never recorded. Keeping them risks introducing noise into the training data since we cannot verify whether the Transit_Days value for these rows is accurate. It is safer to exclude them entirely rather than impute a date.

---

4 SELECT TOOL — TYPE CONVERSION

Select Tool — Change Data Types to Double

After filtering, the Select tool is used to explicitly convert Transit_Days and Cost_Numeric from String to Double. This is a critical step — Alteryx's predictive tools require numeric fields to be stored as the correct data type, not as strings.

TYPE CONVERSION — SELECT TOOL CONFIGURATION

FIELD	ORIGINAL TYPE	NEW TYPE	REASON
`Transit_Days`	String	**Double**	Target variable — must be numeric for regression models
`Cost_Numeric`	String	**Double**	Predictor variable — cleaned from Cost string in Step 2
All other fields	Unchanged	Unchanged	String categoricals (Carrier, Origin_Warehouse) left as-is — handled natively by both model types

How to configure the Select tool:

- Drag the Select tool (Preparation palette) after the Filter tool
- In the configuration pane, find Transit_Days in the field list — click its Type dropdown and change from String to Double
- Find Cost_Numeric — change its Type from String to Double
- All other fields remain unchanged — do not modify Carrier, Origin_Warehouse, or Destination
- Deselect (uncheck) columns to drop: Shipment_ID, Delivery_Date, Shipment_Date, and the original raw Cost column

WHY THE SELECT TOOL AND NOT FORMULA?

The Select tool is the correct Alteryx tool for data type changes and column selection — it is purpose-built for this. Using a Formula tool to cast types is possible but creates unnecessary complexity. The Select tool also lets you drop unwanted columns and rename fields in a single step, keeping the canvas clean.

5 FILTER — REMOVE OUTLIERS

Filter Tool — Transit_Days Range

The target variable Transit_Days has a maximum of 12 days, which is above the expected operational range. We filter to keep only shipments between 1 and 10 days to remove outliers that would skew model training.

FILTER CONDITION

```
[Transit_Days] >= 1 AND [Transit_Days] <= 10
```

- True (T) anchor: clean records with valid Transit_Days — continue downstream
- False (F) anchor: connect to a Browse to review excluded outlier shipments

WHY REMOVE OUTLIERS FROM THE TARGET VARIABLE?

Extreme Transit_Days values (11-12 days) likely represent exceptional cases — lost shipments, customs delays, or data entry errors — that do not reflect normal operations. Training the model on these values causes it to try to explain events it cannot generalise from, resulting in worse predictions for the vast majority of normal shipments.

---

6 FEATURE ENGINEERING & TRAIN/TEST SPLIT

Formula Tool + Create Samples Tool

We create four derived features that capture relationships not visible in the raw data, then split into 70% training and 30% testing using the Create Samples tool.

ENGINEERED FEATURES — FORMULA TOOL

NEW FIELD	TYPE	EXPRESSION	BUSINESS MEANING
`Cost_Per_Mile`	Double	`[Cost_Numeric] / [Distance_miles]`	Shipping cost efficiency — proxy for service level
`Weight_Per_Mile`	Double	`[Weight_kg] / [Distance_miles]`	Load density ratio — captures weight-distance interaction
`Distance_Band`	String	`IIF([Distance_miles]<500,"Short", IIF([Distance_miles]<1500,"Medium","Long"))`	Non-linear distance groups for pattern detection
`Is_Heavy`	Int	`IIF([Weight_kg] > 50, 1, 0)`	Binary flag for heavy shipments needing special handling

CREATE SAMPLES — TRAIN / TEST SPLIT

- Drag Create Samples tool (Preparation palette) after the Formula tool
- Set Estimation Sample to 70% — this becomes the training set
- Set Validation Sample to 30% — this is the hold-out test set
- Set Random Seed = 42 for reproducibility across all team members
- E anchor (Estimation) → feeds both model tools | V anchor (Validation) → feeds both Score tools

> **WHY NO ENCODING FOR CARRIER AND ORIGIN_WAREHOUSE?**
>
> Both Linear Regression and Random Forest in Alteryx use R internally. R handles string categorical columns natively — Linear Regression converts them to dummy variables automatically, and Random Forest splits on factor levels directly. Manual One-Hot encoding adds canvas complexity with zero accuracy benefit for these two model types specifically.

7 MODEL TRAINING & SCORING

Linear Regression + Random Forest + Score Tools

Both models are trained in parallel on the 70% estimation sample. Each trained model is then applied to the 30% validation sample using the Score tool to generate predicted Transit_Days values.

Training Sample 70% → **Linear Regression + Random Forest** → **Score × 2 Test Sample** → **Scored Output**

	LINEAR REGRESSION	RANDOM FOREST
Role	Interpretable baseline	**Primary production model**
Target field	`Transit_Days`	`Transit_Days`
Algorithm	OLS via R lm()	500 decision trees (R randomForest)
Key config	All defaults — Alteryx handles encoding automatically	Number of Trees: 500 | Random Seed: 42 | Variable Importance: ON
Best for	Coefficient interpretation — quantifying each feature's exact effect in days	Accuracy — captures non-linear carrier and distance interactions
Output anchors	O anchor → Score tool R anchor → Coefficients report	O anchor → Score tool R anchor → Variable Importance chart

SCORE TOOL — APPLYING MODELS TO TEST DATA

- Add one Score tool (Predictive palette) per model — two Score tools total
- Connect the model's O anchor to the Score tool's M (model) anchor
- Connect the 30% Validation Sample to the Score tool's D (data) anchor
- Output: all original test columns plus a new 'Score' column = predicted Transit_Days
- Add a Formula tool after each Score output — create field Model_Name = 'Linear Regression' or 'Random Forest'
- Connect both tagged outputs to a Union tool (Preparation palette) — stacks ~1,200 rows total

WHY SCORE ON HELD-OUT TEST DATA — NOT TRAINING DATA?

Measuring accuracy on the training data would only show how well the model memorised what it already saw — not how well it predicts new, unseen shipments. The 30% hold-out set simulates real deployment. Every metric reported in Step 8 reflects true out-of-sample generalisation performance.

8 MODEL COMPARISON & OUTPUT DATA

Model Comparison Tool + Output Data

The Model Comparison tool evaluates both models on the same test set and reports R^2, RMSE, and MAE side by side. The best model's predictions are then exported to two output files.

Union (tagged rows) → **Model Comparison** → **Formula (Residuals)** → **Select (columns)** → **Output Data (.csv + .rds)**

MODEL COMPARISON — CONFIGURATION

- Connect Union output → Model Comparison tool (Predictive palette)
- Target field: Transit_Days | Predicted field: Score | Group by: Model_Name
- Alteryx auto-detects regression (numeric target) — reports R^2, RMSE, MAE per model
- Connect O anchor to Browse to see raw metric numbers before rendering

EXPECTED PERFORMANCE METRICS

METRIC	LINEAR REGRESSION	RANDOM FOREST	INTERPRETATION
R^2	~0.55 – 0.65	**~0.75 – 0.88**	RF explains ~80% of Transit_Days variation
RMSE	~1.2 – 1.5 days	**~0.7 – 1.0 days**	RF predictions off by under 1 day on average
MAE	~0.9 – 1.1 days	**~0.5 – 0.8 days**	Average unsigned error — RF is operationally superior

OUTPUT DATA — FINAL DELIVERABLES

- Add Formula tool after RF Score output — create: Residual = [Transit_Days] - [Score] and Abs_Error = ABS([Transit_Days] - [Score])
- Add Select tool — keep: Distance_miles, Weight_kg, Cost_Numeric, Carrier, Origin_Warehouse, Transit_Days, Score, Residual, Abs_Error
- Output Data tool #1 → file: transit_days_predictions.csv (predictions for the ops team)
- Output Data tool #2 → connect RF model O anchor → file: rf_transit_model.rds (saved model object for scoring future shipments)

DEPLOYMENT — SCORING FUTURE SHIPMENTS WITHOUT RETRAINING

The saved rf_transit_model.rds file contains the fully trained Random Forest model. To score new incoming shipments: create a separate scoring workflow that loads the .rds via Input Data, connects it to a Score tool alongside the new shipment data, and writes predictions to a new output file. No model retraining is required until the RMSE on new data exceeds 1.2 days — schedule a monthly retrain to keep the model fresh as carrier performance evolves.

COMPLETE WORKFLOW SUMMARY

What We Built & What It Delivers

A complete, reproducible Alteryx workflow that takes raw logistics data, cleans and engineers it, trains two models, compares their performance, and produces deployable output files ready to score new shipments at dispatch time.

STEP	TOOL(S)	KEY ACTION	OUTPUT
1	`Input Data, Browse`	Load CSV, profile 11 columns	Raw 2,000-row dataset in workflow
2	`Formula`	Strip '?' from Cost → Cost_Numeric (Double)	Clean numeric cost field
3	`Filter (!IsNull)`	Exclude null Delivery_Date rows	Complete records only
4	`Select`	Change Transit_Days + Cost_Numeric to Double	Correct data types for models
5	`Filter (range)`	Keep Transit_Days 1–10, remove outliers	Clean target variable
6	`Formula + Create Samples`	Engineer 4 features, 70/30 split	Training + test datasets
7	`Linear Reg + RF + Score`	Train both models, score test set	Predicted Transit_Days values
8	`Model Comparison + Output`	Compare R^2/RMSE/MAE, export files	.csv predictions + .rds model

7
Workflow Phases

~80%
R^2 — Random Forest

<1 day
Avg Prediction Error (MAE)

2
Output Files Produced

KEY DESIGN DECISIONS

Status field excluded — data leakage risk (consequence of transit, not predictor). No One-Hot encoding — Linear Regression and Random Forest handle strings natively in R. Median imputation used for Cost nulls — cost data is right-skewed so median is more robust than mean. Random Seed 42 set throughout — ensures reproducible results across all team members. Separate training and scoring workflows — retrain monthly, score every new batch.